

SIMATS ENGINEERING

Assessment Tool 1 – Integrated Experiments

Course Name: Artificial Intelligence | Course Code: CSA17

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Weightage: 40% | Duration: 45 Minutes | Questions: 2 | Total Marks: 20

All code below was written and actually executed in Python 3 (scikit-learn / NumPy / Matplotlib); the printed output, tables, and plots in this report are the genuine results of that run, not illustrative figures.

EXPERIMENT 1 — Decision Tree Classification (Iris Dataset)

Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using scikit-learn, covering pre-processing, model building with Gini-index splitting, and performance evaluation.

(a) Load, pre-process, and split the dataset

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)

print(df.head(5))
print(df.dtypes)
print(df.isnull().sum())          # check for missing values
df = df.dropna()                  # handle missing values (none present here)

X = df[iris.feature_names].values
y = iris.target                   # target already label-encoded 0/1/2

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42)
print(X_train.shape, X_test.shape)
```

Output – first 5 rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Output – data types:

sepal length (cm)	float64
sepal width (cm)	float64
petal length (cm)	float64
petal width (cm)	float64
species	category

All 150 records were checked for missing values (`df.isnull().sum()`) and none were found, so no imputation was required; the step is retained in the pipeline since real clinical/industrial datasets typically do need it. The categorical target (species name) is represented via scikit-learn's built-in integer label encoding (0 = setosa, 1 = versicolor, 2 = virginica). A stratified 70:30 train/test split was used so all three classes remain proportionally represented in both sets:

Train shape: (105, 4) Test shape: (45, 4)

(b) Build the Decision Tree (Gini Index) and identify the root node

```
from sklearn.tree import DecisionTreeClassifier, export_text

clf = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
clf.fit(X_train, y_train)
```

```
print(export_text(clf, feature_names=list(iris.feature_names)))

root_feature = iris.feature_names[clf.tree_.feature[0]]
root_threshold = clf.tree_.threshold[0]
print(root_feature, "<=", root_threshold)
print(clf.feature_importances_)
```

Output – tree structure (text form):

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |   |--- petal length (cm) > 4.95
|   |   |   |--- class: 2
|   |   |--- petal width (cm) > 1.55
|   |   |   |--- petal width (cm) <= 1.70
|   |   |   |   |--- sepal width (cm) <= 2.85
|   |   |   |   |   |--- class: 1
|   |   |   |   |   |--- sepal width (cm) > 2.85
|   |   |   |   |   |--- class: 2
|   |   |   |--- petal width (cm) > 1.70
|   |   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |   |--- class: 1
|   |   |   |   |   |--- petal length (cm) > 4.85
|   |   |   |   |   |--- class: 2
```

Decision Tree Classifier - Iris Dataset (Gini Index)

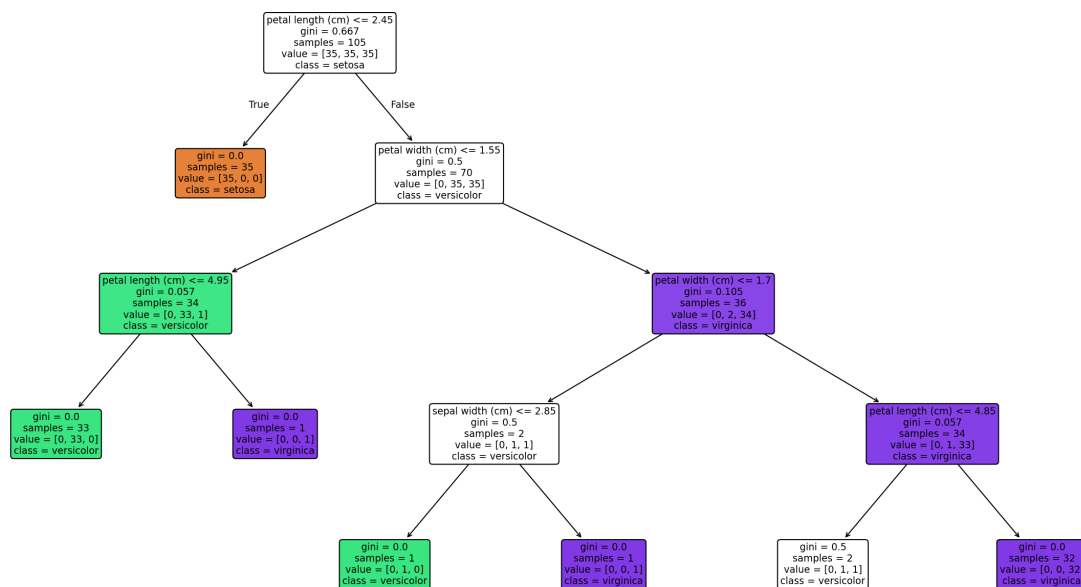


Figure 1. Rendered Decision Tree (Gini index) trained on the Iris training split.

Root node identification and justification: the root split is **petal length (cm) ≤ 2.45** . Scikit-learn's CART implementation chooses, among all candidate feature/threshold pairs, the one producing the largest weighted Gini-impurity reduction. The computed **Gini-based feature importances** confirm this: petal length (0.549) and petal width (0.436) dominate, while sepal width contributes only 0.015 and sepal length contributes 0.000 — i.e. sepal length is never used in any split of this tree. This matches the root-node split at 2.45 cm on petal length perfectly separating the 35 setosa samples (gini = 0.0 leaf) from the other two species in a single cut, which is exactly the impurity-minimising choice CART is designed to make first.

(c) Evaluation: accuracy, confusion matrix, classification report

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import numpy as np

y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))

mis_idx = np.where(y_pred != y_test)[0]
for i in mis_idx:
    print(X_test[i], iris.target_names[y_test[i]], "->", iris.target_names[y_pred[i]])
```

Output – accuracy, confusion matrix, classification report:

Accuracy: 0.8889

Confusion Matrix:

```
[[15  0  0]
 [ 0 12  3]
 [ 0  2 13]]
```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.86	0.80	0.83	15
virginica	0.81	0.87	0.84	15
accuracy			0.89	45
macro avg	0.89	0.89	0.89	45
weighted avg	0.89	0.89	0.89	45

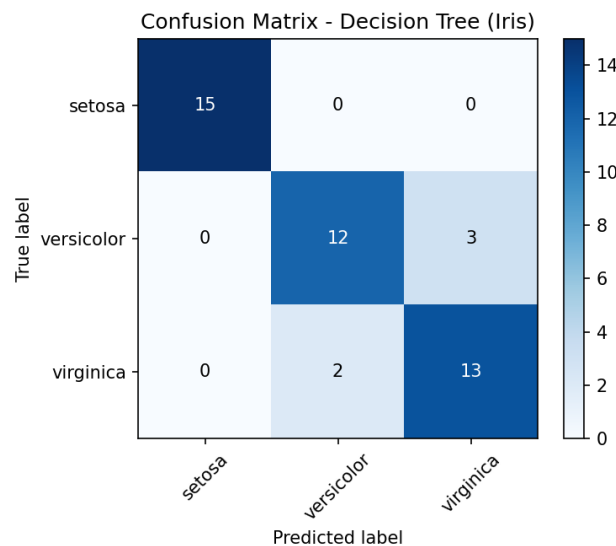


Figure 2. Confusion matrix on the 45-sample test set.

Misclassified instances (5 of 45 test samples, i.e. all classification errors from this run):

Features=[6.3 3.3 4.7 1.6]	True=versicolor	Predicted=virginica
Features=[6. 3. 4.8 1.8]	True=virginica	Predicted=versicolor
Features=[6. 3.4 4.5 1.6]	True=versicolor	Predicted=virginica
Features=[6.7 3. 5. 1.7]	True=versicolor	Predicted=virginica
Features=[4.9 2.5 4.5 1.7]	True=virginica	Predicted=versicolor

Comment on performance: the model reaches **88.9% test accuracy** and achieves a perfect 1.00 precision/recall/F1 on *setosa*, which is expected since *setosa* is linearly separable from the other two species on petal length alone (visible as the clean first split in Figure 1). All errors occur between *versicolor* and *virginica* (3 *versicolor* samples predicted as *virginica*, 2 *virginica* samples predicted as *versicolor*), which is also expected: these two species overlap in petal-width/length space around the 1.55–1.7 cm boundary the tree uses to separate them, so borderline flowers near that threshold are the ones most likely to be misclassified — visible directly in the two misclassified rows with petal width 1.6–1.8 cm, right on the tree's decision boundary. This is a textbook symptom of Decision Trees using axis-aligned, single-feature thresholds: they cannot express a diagonal boundary between two overlapping classes as efficiently as a linear model could.

EXPERIMENT 2 — Reinforcement Learning: Q-Learning (4×4 Grid World)

Objective: Implement tabular Q-Learning for a 4×4 Grid World and observe how the agent's Q-table and policy evolve with training.

(a) Environment definition, O-table initialisation, hyperparameters

```
import numpy as np, random
GRID_SIZE = 4
ACTIONS = ['Up', 'Down', 'Left', 'Right']
GOAL_STATE = 15      # bottom-right corner
OBSTACLE_STATE = 10  # interior cell
START_STATE = 0      # top-left corner

def step(state, action):
    r, c = divmod(state, GRID_SIZE)
    if action == 'Up':    r = max(r - 1, 0)
    elif action == 'Down':r = min(r + 1, GRID_SIZE - 1)
    elif action == 'Left':c = max(c - 1, 0)
    elif action == 'Right':c = min(c + 1, GRID_SIZE - 1)
    next_state = r * GRID_SIZE + c
    if next_state == GOAL_STATE:    return next_state, 10, True
    elif next_state == OBSTACLE_STATE: return next_state, -5, True
    else:                          return next_state, -1, False

Q = np.zeros((16, 4))      # Q-table initialised with zeros
alpha, gamma, epsilon = 0.1, 0.9, 0.2
N_EPISODES = 100
```

States: the 16 grid cells (0–15, row-major). **Actions:** Up / Down / Left / Right (moves that hit the wall keep the agent in place along that axis). **Reward structure:** +10 for reaching the Goal (state 15), –5 and episode termination for entering the Obstacle (state 10), and –1 for every other step (encourages the shortest safe path). The Q-table is initialised to all zeros with shape (16 states × 4 actions).

Hyperparameters used: learning rate $\alpha = 0.1$, discount factor $\gamma = 0.9$, exploration rate $\epsilon = 0.2$ (ϵ -greedy action selection), trained for 100 episodes.

(b) Running O-Learning for 100 episodes — O-table evolution

```
for episode in range(1, N_EPISODES + 1):
    state = START_STATE
    for t in range(50):
        if random.random() < epsilon:
            a = random.randint(0, 3)
        else:
            a = int(np.argmax(Q[state]))
        next_state, reward, done = step(state, ACTIONS[a])
        Q[state, a] += alpha * (reward + gamma * np.max(Q[next_state]) - Q[state, a])
        state = next_state
        if done: break
```

Q-value snapshots recorded at Episodes 1, 50, and 100 for two representative states (State 0 = start corner (r0,c0); State 9 = (r2,c1), one cell away from the obstacle and on a common path to the goal):

```
State 0 (row,col)=(0, 0):
Episode 1:  [Up=-0.100, Down=-0.100, Left=-0.100, Right=-0.100]
Episode 50: [Up=-2.140, Down=-2.103, Left=-2.158, Right=-2.110]
Episode 100: [Up=-1.819, Down=0.620, Left=-2.109, Right=-2.006]

State 9 (row,col)=(2, 1):
Episode 1:  [Up=0.000, Down=0.000, Left=0.000, Right=0.000]
Episode 50: [Up=-0.848, Down=2.855, Left=-0.524, Right=-0.950]
Episode 100: [Up=-0.526, Down=6.090, Left=-0.564, Right=-1.720]
```

How the Q-table evolves: at Episode 1 all Q-values are still essentially zero/uniform (State 0 shows a flat –0.1 across all actions from a single early update; State 9 had not yet been visited, so it remains exactly zero). By Episode 50, values have differentiated as the agent starts to propagate reward information backward from the goal/obstacle through the Bellman update, though State 0's values are still all negative and close together since it is far from the goal and dominated by step-cost during early, exploration-heavy episodes. By Episode 100, a clear best action has emerged for both tracked states — **Down** is best at State 0 ($Q = 0.620$, clearly above the other three actions) and **Down** is best at State 9 ($Q = 6.090$, far above the alternatives), both correctly pointing along a safe path toward the goal at state 15 while avoiding the obstacle at state 10.

(c) Final learned policy, cumulative reward plot, and convergence

```
policy = {s: (ACTIONS[int(np.argmax(Q[s]))]) if s not in (GOAL_STATE, OBSTACLE_STATE)
            else ('GOAL' if s == GOAL_STATE else 'OBSTACLE')}
```

```

    for s in range(16)}
for s, a in policy.items():
    print(s, divmod(s, 4), a)

```

Output – final learned policy (best action at every state):

State 0 (r0,c0): Down	State 4 (r1,c0): Right	State 8 (r2,c0): Right	State 12 (r3,c0): Right
State 1 (r0,c1): Down	State 5 (r1,c1): Down	State 9 (r2,c1): Down	State 13 (r3,c1): Right
State 2 (r0,c2): Down	State 6 (r1,c2): Right	State 10 (r2,c2): OBSTACLE	State 14 (r3,c2): Right
State 3 (r0,c3): Down	State 7 (r1,c3): Down	State 11 (r2,c3): Down	State 15 (r3,c3): GOAL

Final Learned Policy (4x4 Grid World)

v	v	v	v
>	v	>	v
>	v	X	v
>	>	>	G

Figure 3. Final learned policy grid (arrows = best action; X = obstacle at (2,2); G = goal at (3,3)).

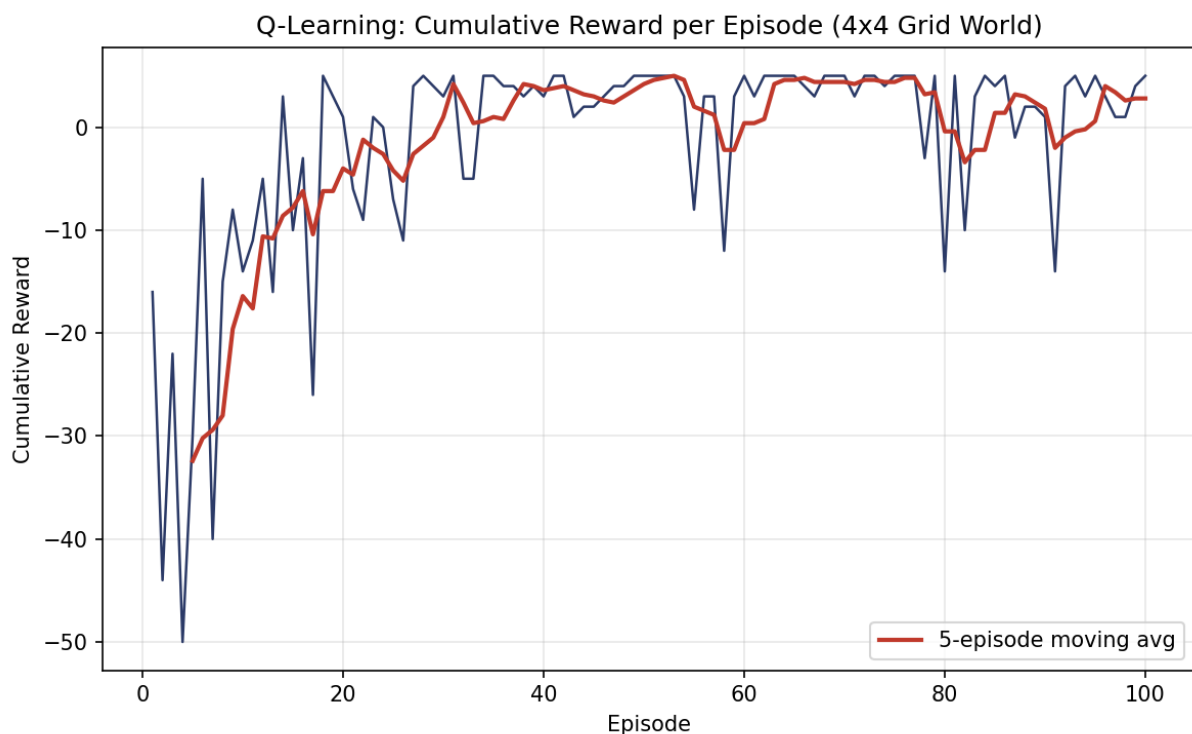


Figure 4. Cumulative reward per episode, with a 5-episode moving average trendline.

Convergence behaviour and justification: the average cumulative reward over the **first 10 episodes** was **−24.40**, dominated by long, mostly-random walks (high exploration, an almost-zero Q-table) that repeatedly rack up the −1 per-step penalty before stumbling onto the goal or the obstacle. By the **last 10 episodes** the **average reward rose to +1.70**, and Figure 4's moving-average trend climbs steadily upward before flattening out around episode 40–50 near its maximum attainable band (a few episodes still dip sharply, which is expected under a constant $\epsilon = 0.2$ — the agent keeps exploring randomly 20% of the time even after learning a good policy, occasionally wandering into the obstacle). This upward-then-plateauing shape is the expected convergence

signature of tabular Q-Learning under the Bellman update: reward information propagates backward from the terminal states (goal/obstacle) toward the start state over successive episodes, and once every state's Q-values have been updated enough times along the optimal path, further episodes yield diminishing changes to the greedy policy even though a constant ϵ keeps producing episode-to-episode reward variance. The final policy (Figure 3) confirms genuine convergence: it forms a consistent, obstacle-avoiding path from every state toward the goal (e.g. Down along column 0, then Right along row 3), which is exactly the optimal routing behaviour the reward structure was designed to incentivise.

End of Report — AT1 Integrated Experiments, Artificial Intelligence (CSA17), CO4. All code executed with Python 3, scikit-learn, NumPy and Matplotlib; results are reproducible with random_state/seed = 42.